

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

--- Clustering ---

```
import pandas as pd

# Path to your Drive folder
base = "/content/drive/MyDrive/Dat Capstone"

# Load both CSVs
products = pd.read_csv(f"{base}/cleaned_makeup_products.csv")
reviews = pd.read_csv(f"{base}/cleaned_makeup_reviews.csv")

# Check shapes and preview
print("Products shape:", products.shape)
print("Reviews shape:", reviews.shape)

display(products.head())
display(reviews.head())
```

```
/tmp/ipython-input-2687498157.py:8: DtypeWarning: Columns (15,16,17,18,19,20) have mixed types. Specify dtype option
reviews = pd.read_csv(f"{base}/cleaned_makeup_reviews.csv")
Products shape: (1373, 35)
Reviews shape: (314029, 27)
```

	product_link_id	product_link	category	item_id	product_name	brand	price	num_shades	rating	num_reviews
0	8	https://www.ulta.com/p/futurist-skin-tint-seru...	NaN	2612458.0	Futurist Skin Tint Serum Foundation SPF 20	Find your shade	55.0	NaN	4.4	1
1	13	https://www.ulta.com/p/dior-forever-fluid-skin...	NaN	2605037.0	Dior Forever Fluid Skin Glow Foundation	Dior	57.0	42.0	4.5	1
2	22	https://www.ulta.com/p/barepro-24hr-wear-skin-...	NaN	2619782.0	BAREPRO 24HR Wear Skin-Perfecting Matte Liquid...	bareMinerals	44.0	NaN	4.1	1
3	32	https://www.ulta.com/p/futurist-hydra-rescue-m...	NaN	2559137.0	Futurist Hydra Rescue Moisturizing Foundation ...	Estée Lauder	55.0	NaN	4.4	1
4	15	https://www.ulta.com/p/mini-cc-cream-with-spf-...	Foundation	2603710.0	Mini CC+ Cream with SPF 50+	IT Cosmetics	22.0	NaN	4.3	1

5 rows x 35 columns

	unique_review_id	product_link_id	review_id	type	id	ugc_id	legacy_id	internal_review_id	headline	num_likes
0	1	8	521867580	image	521867563.0	NaN	NaN	-1	2C3 Fresco	1
1	2	8	485614895	image	485614799.0	NaN	NaN	-1	Glowing Skin!	1
2	3	8	485250956	image	485250927.0	NaN	NaN	-1	Love it	Cgb
3	4	8	485086522	video	485086531.0	NaN	NaN	-1	Horrible allergic reaction	1
4	5	8	482905142	image	482905180.0	NaN	NaN	-1	Futurist Skin tint	Sant

5 rows x 27 columns

```
# --- Core ---
import pandas as pd
import numpy as np

# --- Modeling & evaluation ---
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples

# --- Utils & plots ---
import matplotlib.pyplot as plt
import textwrap
import os
from pathlib import Path
np.random.seed(42)
```

```
base = "/content/drive/MyDrive/Dat Capstone"          # <-- change if needed
csv_path = f"{base}/cleaned_makeup_products.csv"

# low_memory=False avoids the mixed dtype warning, and we'll coerce numeric below
df = pd.read_csv(csv_path, low_memory=False)

print("Shape:", df.shape)
display(df.head(3))

# optional: peek at dtypes to see text vs numeric
df.info()
```

Shape: (1373, 35)

	product_link_id	product_link	category	item_id	product_name	brand	price	num_shades	rating	num_reviews
0	8	https://www.ulta.com/p/futurist-skin-tint-seru...	NaN	2612458.0	Futurist Skin Tint Serum Foundation SPF 20	Find your shade	55.0	NaN	4.4	
1	13	https://www.ulta.com/p/dior-forever-fluid-skin...	NaN	2605037.0	Dior Forever Fluid Skin Glow Foundation	Dior	57.0	42.0	4.5	
2	22	https://www.ulta.com/p/barepro-24hr-wear-skin...	NaN	2619782.0	BAREPRO 24HR Wear Skin-Perfecting Matte Liquid...	bareMinerals	44.0	NaN	4.1	

3 rows x 35 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1373 entries, 0 to 1372
```

Data columns (total 35 columns):

#	Column	Non-Null Count	Dtype
0	product_link_id	1373 non-null	int64
1	product_link	1373 non-null	object
2	category	1351 non-null	object
3	item_id	853 non-null	float64
4	product_name	1373 non-null	object
5	brand	930 non-null	object
6	price	1373 non-null	float64
7	num_shades	64 non-null	float64
8	rating	1232 non-null	float64
9	num_reviews	1372 non-null	float64
10	description	1373 non-null	object
11	pros	755 non-null	object
12	cons	675 non-null	object
13	best_uses	643 non-null	object
14	describe_yourself	690 non-null	object
15	review_star_1	1373 non-null	int64
16	review_star_2	1373 non-null	int64
17	review_star_3	1373 non-null	int64
18	review_star_4	1373 non-null	int64
19	review_star_5	1373 non-null	int64
20	rating_star_1	1373 non-null	int64
21	rating_star_2	1373 non-null	int64
22	rating_star_3	1373 non-null	int64
23	rating_star_4	1373 non-null	int64
24	rating_star_5	1373 non-null	int64
25	rating_count	1373 non-null	int64
26	review_count	1373 non-null	int64
27	average_rating	1373 non-null	float64
28	recommended_ratio	1373 non-null	float64
29	native_review_count	1373 non-null	int64
30	native_sampling_review_count	0 non-null	float64
31	native_community_content_review_count	1373 non-null	int64
32	syndicated_review_count	1373 non-null	int64
33	faceoff_negative	1244 non-null	object
34	faceoff_positive	1244 non-null	object

2) define columns & clean (IDs/text dropped, numerics kept) – SAFE

```
expected_cols = [
    "product_link_id","product_link","category","item_id","product_name","brand","price","num_shades",
    "rating","num_reviews","description","pros","cons","best_uses","describe_yourself",
    "review_star_1","review_star_2","review_star_3","review_star_4","review_star_5",
    "rating_star_1","rating_star_2","rating_star_3","rating_star_4","rating_star_5",
    "rating_count","review_count","average_rating","recommended_ratio",
    "native_review_count","native_sampling_review_count",
    # "native_community_count", # <-- not present in your file; comment out or let code skip missing
    "syndicated_review_count","faceoff_negative","faceoff_positive"
]
```

```
present_cols = [c for c in expected_cols if c in df.columns]
missing_cols = sorted(set(expected_cols) - set(df.columns))
print("Missing (ignored):", missing_cols)
```

drop ID/url/text/meta for clustering (keep for reporting later)

```
drop_cols = [
    "product_link_id","product_link","item_id","product_name",
    "description","pros","cons","best_uses","describe_yourself","brand","category"
]
drop_cols = [c for c in drop_cols if c in df.columns] # keep only those that exist
```

```
# candidate numeric features (that actually exist)
num_cols = [c for c in present_cols if c not in drop_cols]
print("Candidate numeric columns:", num_cols)

# coerce everything in num_cols to numeric where possible
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

numeric_df = df.loc[:, num_cols].copy()
numeric_df = numeric_df.dropna(axis=1, how="all")

print("Numeric columns retained:", list(numeric_df.columns))
display(numeric_df.describe().T.round(3).head(15))
```

```
Missing (ignored): []
Candidate numeric columns: ['price', 'num_shades', 'rating', 'num_reviews', 'review_star_1', 'review_star_2', 'review_star_3', 'review_star_4', 'review_star_5', 'rating_star_1', 'rating_star_2', 'rating_star_3', 'rating_star_4', 'rating_star_5', 'rating_count']
Numeric columns retained: ['price', 'num_shades', 'rating', 'num_reviews', 'review_star_1', 'review_star_2', 'review_star_3', 'review_star_4', 'review_star_5', 'rating_star_1', 'rating_star_2', 'rating_star_3', 'rating_star_4', 'rating_star_5', 'rating_count']
```

	count	mean	std	min	25%	50%	75%	max
price	1373.0	26.744	15.341	1.5	12.99	26.0	38.0	85.0
num_shades	64.0	21.516	18.815	2.0	3.00	20.0	36.0	67.0
rating	1232.0	4.309	0.383	1.8	4.20	4.4	4.6	4.9
num_reviews	1372.0	1060.634	2869.369	1.0	53.00	290.5	886.5	37663.0
review_star_1	1373.0	235.356	1093.094	0.0	4.00	25.0	120.0	30485.0
review_star_2	1373.0	237.086	852.624	0.0	5.00	27.0	143.0	18736.0
review_star_3	1373.0	160.392	749.831	0.0	4.00	23.0	99.0	18736.0
review_star_4	1373.0	162.136	879.618	0.0	4.00	22.0	88.0	21758.0
review_star_5	1373.0	265.355	1231.178	0.0	5.00	28.0	159.0	30485.0
rating_star_1	1373.0	235.712	1093.341	0.0	4.00	25.0	121.0	30485.0
rating_star_2	1373.0	237.204	852.617	0.0	5.00	27.0	143.0	18736.0
rating_star_3	1373.0	160.471	749.823	0.0	4.00	23.0	99.0	18736.0
rating_star_4	1373.0	162.257	879.618	0.0	4.00	22.0	88.0	21758.0
rating_star_5	1373.0	265.985	1231.463	0.0	5.00	28.0	159.0	30485.0
rating_count	1373.0	1061.629	2868.790	0.0	53.00	291.0	891.0	37663.0

```
import math
import matplotlib.pyplot as plt

cols = numeric_df.columns
n = len(cols)
plots_per_fig = 4 # 2x2 layout → 4 plots per figure

for start in range(0, n, plots_per_fig):
    end = min(start + plots_per_fig, n)
    subset = cols[start:end]

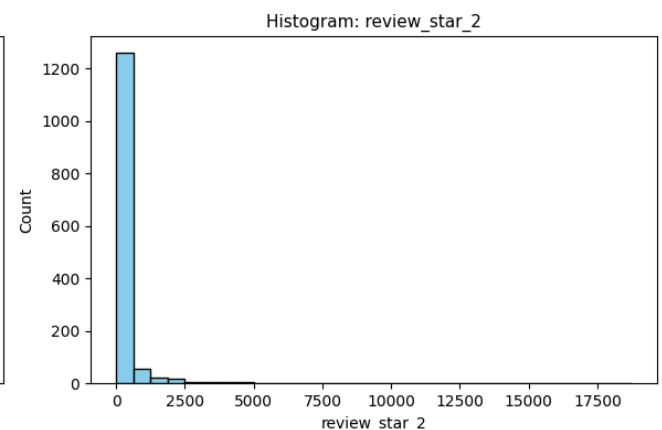
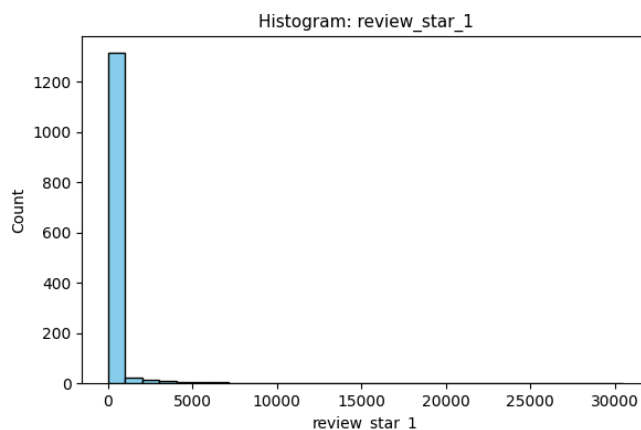
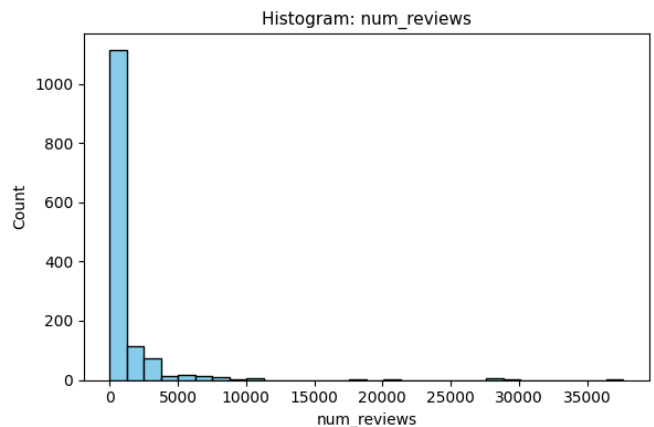
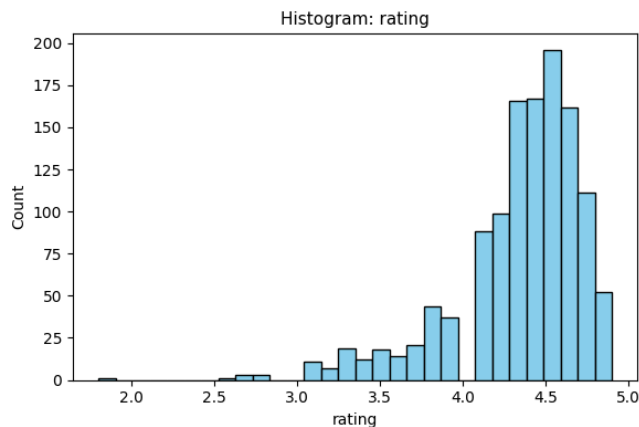
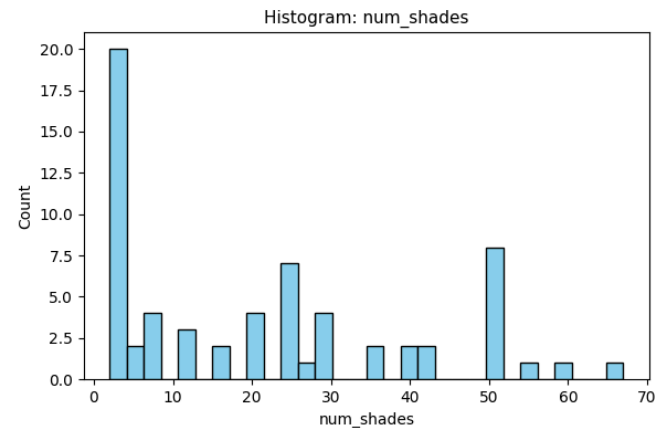
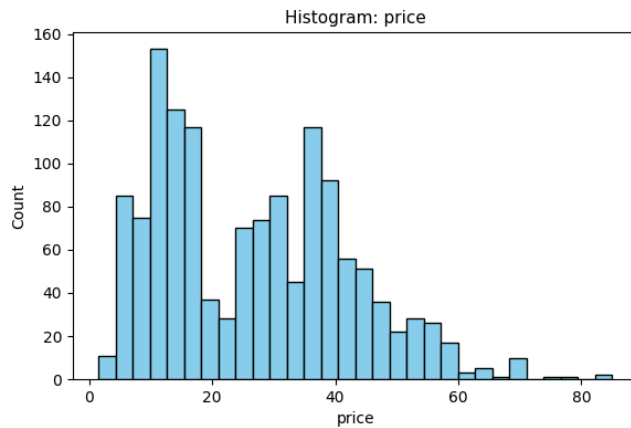
    nrows, ncols = 2, 2
    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(12, 8))
    axes = axes.flatten()

    for i, c in enumerate(subset):
        ax = axes[i]
        ax.hist(numeric_df[c].dropna(), bins=30, color="skyblue", edgecolor="black")
        ax.set_title(f"Histogram: {c}", fontsize=11)
        ax.set_xlabel(c)
        ax.set_ylabel("Count")

    # hide unused subplots if fewer than 4 left
    for j in range(i+1, len(axes)):
        axes[j].set_visible(False)

    plt.tight_layout()
    fig_num = start // plots_per_fig + 1
    plt.savefig(f"{base}/basic_eda_part{fig_num}.png", dpi=160, bbox_inches="tight")
```

plt.show()



Step 4: Handle missing values

```
X = numeric_df.copy()
X = X.fillna(X.median(numeric_only=True))

# drop columns with no variance (e.g., same value for all)
low_var = X.nunique() <= 1
if low_var.any():
    print("Dropping near-constant columns:", list(X.columns[low_var]))
    X = X.loc[:, ~low_var]
```

#Step 5: Log-transform skewed variables

```
import numpy as np
```

```
skew_candidates = [c for c in X.columns if (X[c].max() > 100) and (X[c].min() >= 0)]
print("Skew candidates (log1p transform):", skew_candidates)
```

```
X_log = X.copy()
for c in skew_candidates:
```

```
X_log[c] = np.log1p(X[c]) # log1p handles zeros
```

```
Skew Candidates (log1p transform): ['num_reviews', 'review_star_1', 'review_star_2', 'review_star_3', 'review_star_4']
```

```
#Step 6: Standardize data (scaling)
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = pd.DataFrame(scaler.fit_transform(X_log), columns=X_log.columns, index=X_log.index)
```

```
display(X_scaled.head(3))
```

	price	num_shades	rating	num_reviews	review_star_1	review_star_2	review_star_3	review_star_4	review_star_5
0	1.842533	-0.017474	0.224079	0.727766	1.138838	0.499451	1.590673	-0.121061	-0.015121
1	1.972950	5.424115	0.499021	1.156053	0.135569	0.297456	0.840843	1.599373	1.831081
2	1.125236	-0.017474	-0.600748	1.360653	2.085232	1.163040	1.158108	1.215284	1.319713

```
#Step 7: Elbow method
```

```
from sklearn.cluster import KMeans
```

```
inertias = []
```

```
ks = range(2, 13)
```

```
for k in ks:
```

```
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
```

```
    km.fit(X_scaled)
```

```
    inertias.append(km.inertia_)
```

```
plt.figure(figsize=(6,4))
```

```
plt.plot(list(ks), inertias, marker="o")
```

```
plt.xlabel("k")
```

```
plt.ylabel("Within-Cluster Sum of Squares (Inertia)")
```

```
plt.title("Elbow Curve for K-Means")
```

```
plt.grid(True, alpha=.3)
```

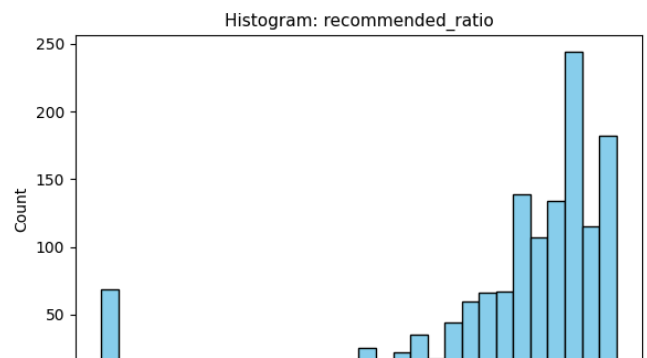
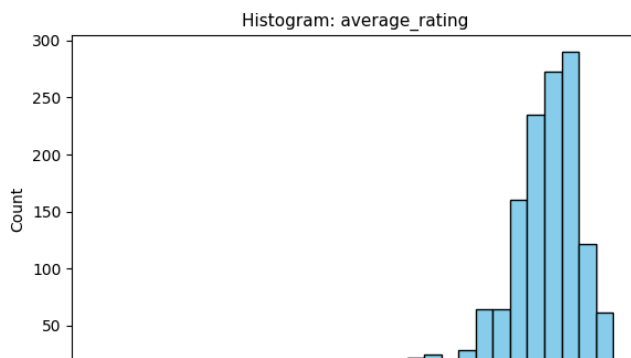
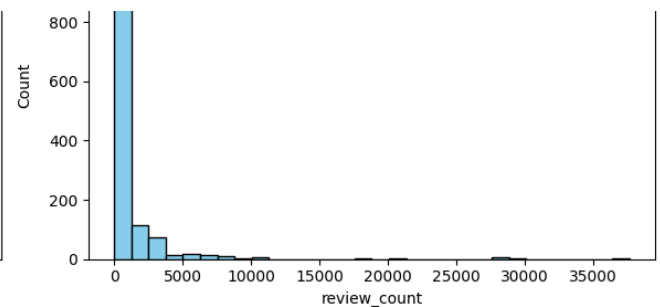
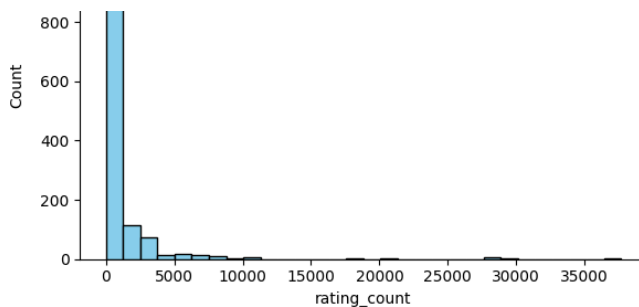
```
plt.tight_layout()
```

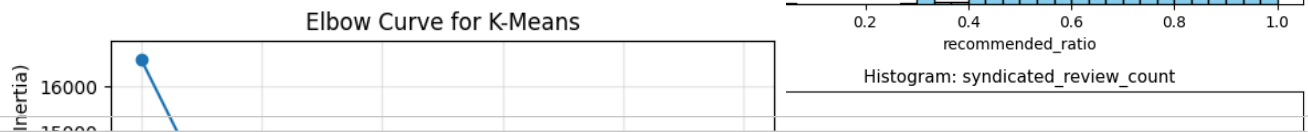
```
plt.savefig("/content/drive/MyDrive/Data Capstone/elbow_curve_for_kmeans.png",  
            dpi=300, bbox_inches="tight")
```

```
plt.show()
```

```
for k, s in zip(ks, inertias):
```

```
    print(f"k={k:2d} inertia={s:,.0f}")
```





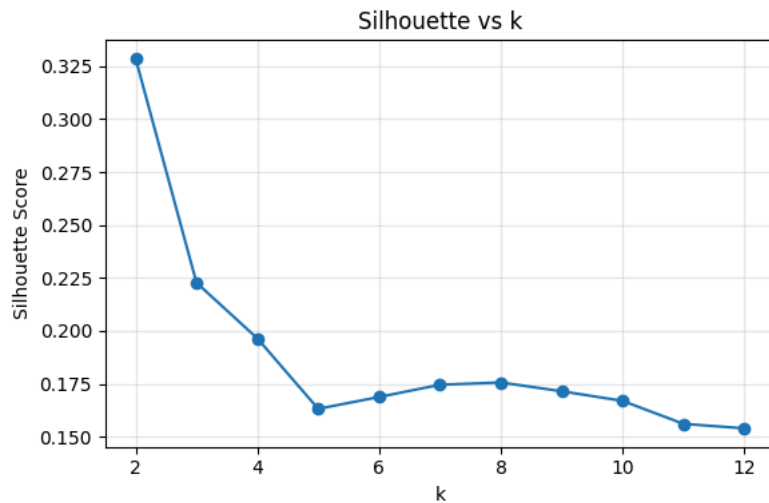
#Step 8: Silhouette scores

```
from sklearn.metrics import silhouette_score

sil_scores = []
for k in ks:
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = km.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)

plt.figure(figsize=(6,4))
plt.plot(list(ks), sil_scores, marker="o")
plt.xlabel("k")
plt.ylabel("Silhouette Score")
plt.title("Silhouette vs k")
plt.grid(True, alpha=.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouette_vs_k.png",
            dpi=300, bbox_inches="tight")
plt.show()

for k, s in zip(ks, sil_scores):
    print(f"k={k:2d}  silhouette={s:.3f}")
```



```
k= 2  silhouette=0.329
k= 3  silhouette=0.223
k= 4  silhouette=0.196
k= 5  silhouette=0.163
k= 6  silhouette=0.169
k= 7  silhouette=0.175
k= 8  silhouette=0.176
k= 9  silhouette=0.171
k=10  silhouette=0.167
k=11  silhouette=0.156
k=12  silhouette=0.154
```

Choose the best number of clusters based on your plots
k_best = 4 # <-- change this if another value looks better

```
from sklearn.cluster import KMeans

# Fit K-Means with chosen k
km_final = KMeans(n_clusters=k_best, n_init=50, random_state=42)
labels = km_final.fit_predict(X_scaled)

# Attach labels to original dataframe
df_clusters = df.copy()
df_clusters["cluster"] = labels
```

```
print("Cluster sizes:")
print(df_clusters["cluster"].value_counts().sort_index())

# quick look
display(df_clusters[["product_name", "brand", "price", "average_rating", "cluster"]].head(10))
```

Cluster sizes:

```
cluster
0    146
1    582
2    312
3    333
```

Name: count, dtype: int64

	product_name	brand	price	average_rating	cluster
0	Futurist Skin Tint Serum Foundation SPF 20	Find your shade	55.0	4.36	1
1	Dior Forever Fluid Skin Glow Foundation	Dior	57.0	4.53	2
2	BAREPRO 24HR Wear Skin-Perfecting Matte Liquid...	bareMinerals	44.0	4.09	2
3	Futurist Hydra Rescue Moisturizing Foundation ...	Estée Lauder	55.0	4.36	2
4	Mini CC+ Cream with SPF 50+	IT Cosmetics	22.0	4.30	2
5	Teint Idole Ultra Wear 24H Full Coverage Found...	Lancôme	57.0	4.45	2
6	Studio Fix Powder Plus Foundation Makeup	MAC	44.0	4.56	2
7	Light Reflecting Advanced Skincare Foundation	NARS	52.0	4.47	2
8	Studio Fix Fluid SPF15 24HR Matte Foundation +...	MAC	44.0	4.33	1
9	Born This Way Natural Finish Longwear Liquid F...	Too Faced	46.0	4.32	2

```
# STEP 9: Final K-Means model
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples
import matplotlib.pyplot as plt
import numpy as np

k_best = 4 # final chosen k
km_final = KMeans(n_clusters=k_best, n_init=50, random_state=42)
labels = km_final.fit_predict(X_scaled)

df_clusters = df.copy()
df_clusters["cluster"] = labels

print("Cluster sizes:")
print(df_clusters["cluster"].value_counts().sort_index())
```

Cluster sizes:

```
cluster
0    146
1    582
2    312
3    333
```

Name: count, dtype: int64

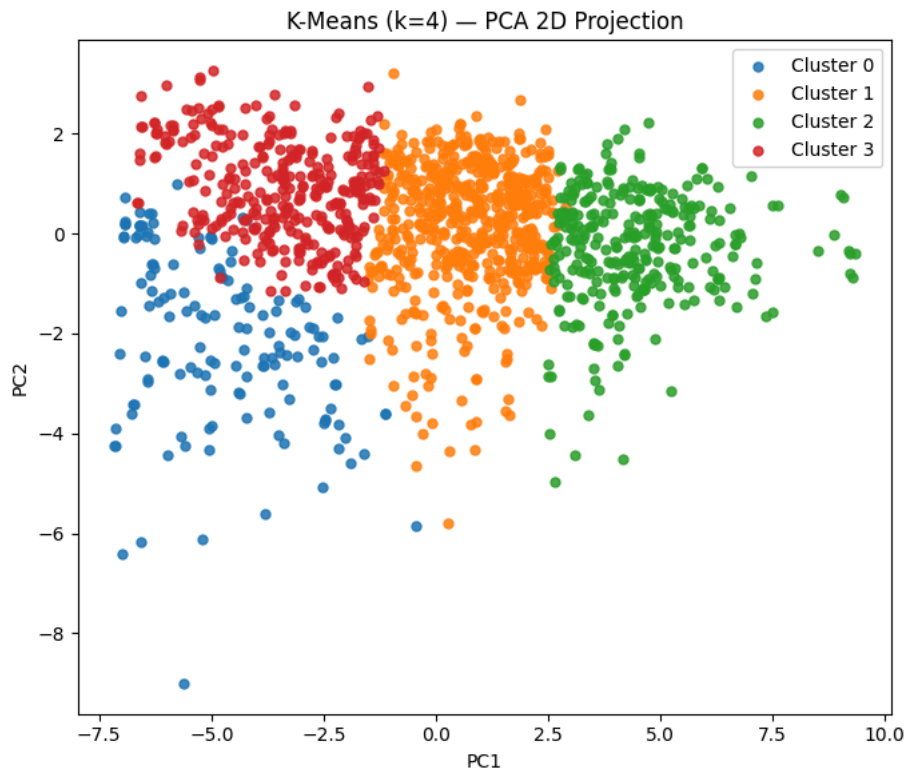
```
#Step 10 – PCA 2D visualization
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2, random_state=42)
proj2 = pca.fit_transform(X_scaled)

plt.figure(figsize=(7,6))
for c in range(k_best):
    idx = (labels == c)
    plt.scatter(proj2[idx, 0], proj2[idx, 1], s=25, alpha=.85, label=f"Cluster {c}")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(f"K-Means (k={k_best}) – PCA 2D Projection")
plt.legend()
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/pca_2d_projection.png",
            dpi=300, bbox_inches="tight")
```

```
plt.show()

print("Explained variance ratio:", pca.explained_variance_ratio_)
```

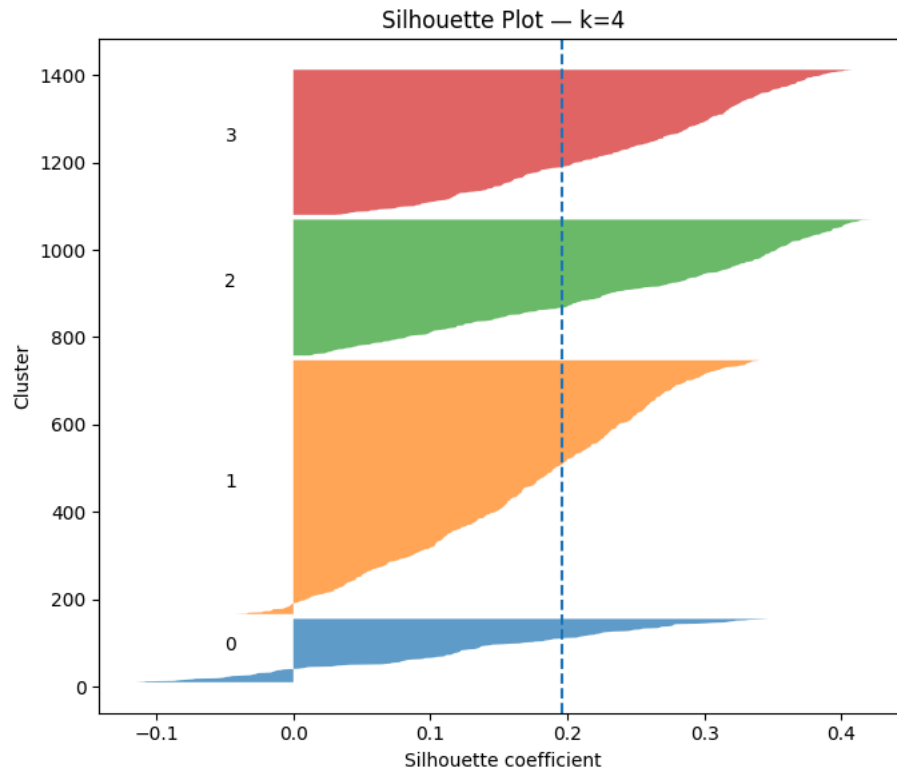


```
from sklearn.metrics import silhouette_samples
import numpy as np
import matplotlib.pyplot as plt

sample_sil = silhouette_samples(X_scaled, labels)

y_lower = 10
plt.figure(figsize=(7,6))
for i in range(k_best):
    ith = sample_sil[labels == i]
    ith.sort()
    size_i = ith.shape[0]
    y_upper = y_lower + size_i
    plt.fill_betweenx(np.arange(y_lower, y_upper), 0, ith, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5*size_i, str(i))
    y_lower = y_upper + 10

plt.axvline(np.mean(sample_sil), linestyle="--")
plt.xlabel("Silhouette coefficient")
plt.ylabel("Cluster")
plt.title(f"Silhouette Plot - k={k_best}")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouette_plot_k-4vs_k.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



```
import matplotlib.pyplot as plt
import math

# assuming you already have X_scaled or your feature data
kmeans = KMeans(n_clusters=k_best, random_state=42)
kmeans.fit(X_scaled)

# convert cluster centers to a DataFrame
centers_orig = pd.DataFrame(kmeans.cluster_centers_, columns=X_scaled.columns)
out_dir = Path(".") # current directory

# set how many plots per page
cols_per_fig = 3
rows_per_fig = 3
plots_per_fig = cols_per_fig * rows_per_fig

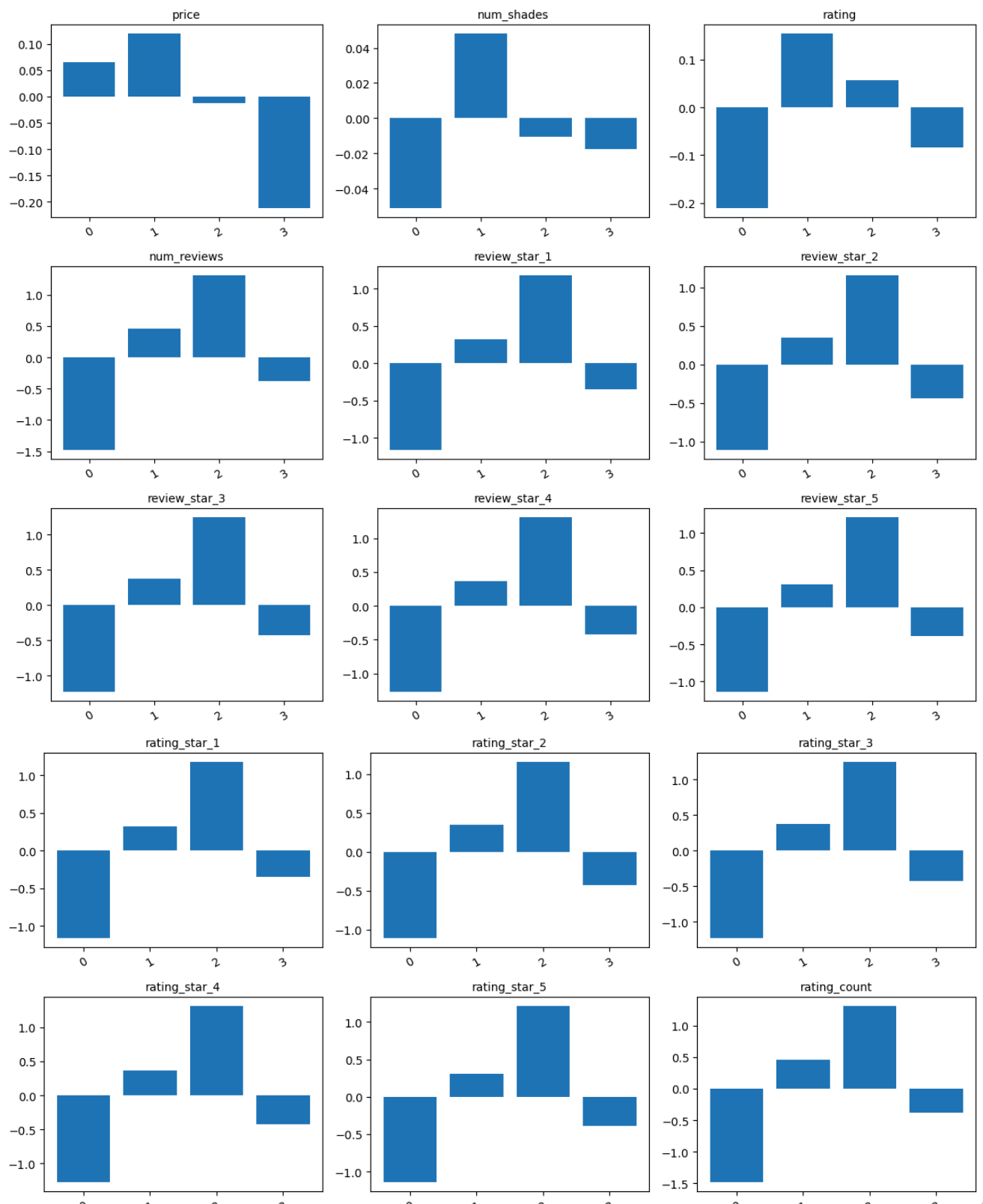
cols = centers_orig.columns.tolist()
num_plots = len(cols)
num_figs = math.ceil(num_plots / plots_per_fig)

for f in range(num_figs):
    start = f * plots_per_fig
    end = start + plots_per_fig
    subset = cols[start:end]

    fig, axes = plt.subplots(rows_per_fig, cols_per_fig, figsize=(12, 9))
    axes = axes.flatten()

    for i, col in enumerate(subset):
        axes[i].bar(centers_orig.index, centers_orig[col].values)
        axes[i].set_title(col, fontsize=10)
        axes[i].tick_params(axis='x', rotation=30)
        axes[i].set_ylabel("")
    # turn off unused subplots
    for j in range(len(subset), len(axes)):
        axes[j].axis("off")

    plt.tight_layout()
    plt.savefig(out_dir / f"centers_grid_summary_{f+1}.png", dpi=160)
    plt.show()
```

```
# Use the exact features that went into modeling
features_used = list(X.columns)
```

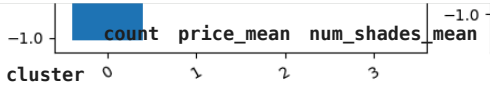
```
profile_count = df_clusters["cluster"].value_counts().sort_index().rename("count")
profile_mean = df_clusters.groupby("cluster")[features_used].mean(numeric_only=True).add_suffix("_mean")
profile_med = df_clusters.groupby("cluster")[features_used].median(numeric_only=True).add_suffix("_median")
```

```
profile = pd.concat([profile_count, profile_mean, profile_med], axis=1).sort_index()
display(profile.round(2))
```

```
# save profile and labeled dataset
profile.to_csv(out_dir / "06_cluster_profile.csv")
(df_clusters
```

```
.to_csv(out_dir / "products_with_clusters.csv", index=False))
```

```
print("Saved:")
print("-", out_dir / "04_pca_2d.png")
print("-", out_dir / "05_silhouette_bars.png")
print("-", out_dir / "centers_logspace.csv")
print("-", out_dir / "centers_original_units.csv")
print("-", out_dir / "06_cluster_profile.csv")
print("-", out_dir / "products_with_clusters.csv")
```



cluster	count	price_mean	num_shades_mean	rating_mean	num_reviews_mean	review_star_1_mean	review_star_2_mean	review_star_3_mean
0	146	21.48	5.00	3.64	20.71	4.98	5.25	
1	582	26.83	26.90	4.35	459.12	102.45	123.93	
2	312	27.00	19.73	4.33	3746.43	829.77	796.27	
3	333	28.67	17.25	4.45	48.33	11.72	12.59	

4 rows x 41 columns

Saved:

- 04_pca_2d.png
- 05_silhouette_bars.png
- centers_logspace.csv
- centers_original_units.csv
- 06_cluster_profile.csv
- products_with_clusters.csv

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # noqa: F401 (for 3D projection)

pca3 = PCA(n_components=3, random_state=42)
proj3 = pca3.fit_transform(X_scaled)

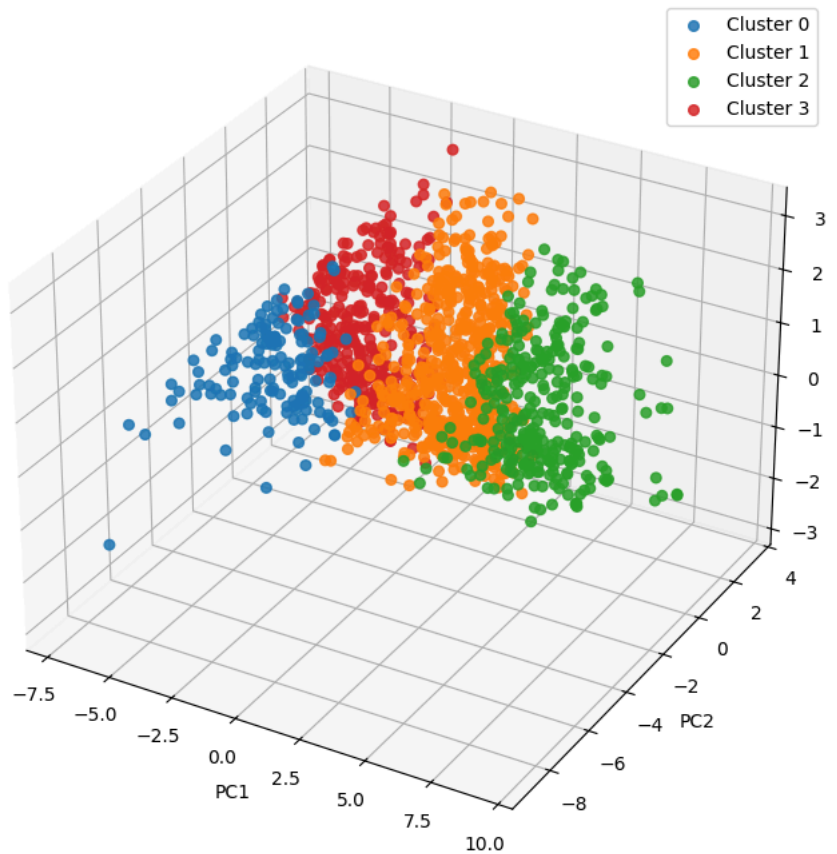
fig = plt.figure(figsize=(8,7))
ax = fig.add_subplot(111, projection="3d")

for c in range(k_best):
    idx = labels == c
    ax.scatter(
        proj3[idx,0], proj3[idx,1], proj3[idx,2],
        s=30, alpha=.85, label=f"Cluster {c}"
    )

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("3D PCA Projection - K-Means Clusters")
ax.legend()
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/3d_pca_projection_k-means.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Explained variance ratio (3 components):", pca3.explained_variance_ratio_)
```

3D PCA Projection — K-Means Clusters



Explained variance ratio (3 components): [0.57745109 0.10966273 0.06119383]

```
!pip -q install fastcluster
```

194.1/194.1 kB 6.1 MB/s eta 0:00:00

```
## Hierarchal Clustering

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.decomposition import PCA

from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

# Optional (faster linkage on big data if installed)
try:
    import fastcluster
except:
    fastcluster = None

np.random.seed(42)

base = "/content/drive/MyDrive/Dat Capstone"
out_dir = Path(base) / "hierarchical_outputs"
out_dir.mkdir(parents=True, exist_ok=True)
```

```
#Load data
```

```
df = pd.read_csv(f"{base}/cleaned_makeup_products.csv", low_memory=False)
print("Shape:", df.shape)
df.head(3)
```

```
Shape: (1373, 35)
```

	product_link_id	product_link	category	item_id	product_name	brand	price	num_shades	rating	num_reviews
0	8	https://www.ulta.com/p/futurist-skin-tint-seru...	NaN	2612458.0	Futurist Skin Tint Serum Foundation SPF 20	Find your shade	55.0	NaN	4.4	
1	13	https://www.ulta.com/p/dior-forever-fluid-skin...	NaN	2605037.0	Dior Forever Fluid Skin Glow Foundation	Dior	57.0	42.0	4.5	
2	22	https://www.ulta.com/p/barepro-24hr-wear-skin...	NaN	2619782.0	BAREPRO 24HR Wear Skin-Perfecting Matte Liquid...	bareMinerals	44.0	NaN	4.1	

```
3 rows x 35 columns
```

```
#4) Impute, transform, and build a preprocessing pipeline
```

```
# numeric = number dtypes
num_cols = df.select_dtypes(include=["number"]).columns.tolist()
# categorical = object/category/bool (cast bool to category for OHE)
cat_cols = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

# Work on a copy to keep originals
work = df.copy()

# Simple median impute for numeric
work[num_cols] = work[num_cols].fillna(work[num_cols].median(numeric_only=True))

# Build a ColumnTransformer
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore", drop="first")
pre = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", ohe, cat_cols)
    ],
    remainder="drop"
)

# Fit/transform to get final feature matrix
X = pre.fit_transform(work)

# Get feature names for later (optional, nice for profiling)
num_names = num_cols
cat_names = []
if len(cat_cols):
    cat_names = list(pre.named_transformers_["cat"].get_feature_names_out(cat_cols))
feature_names = num_names + cat_names

print("Final feature matrix shape:", X.shape)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1101: RuntimeWarning: invalid value encountered in d
    updated_mean = (last_sum + new_sum) / updated_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1106: RuntimeWarning: invalid value encountered in d
    T = new_sum / new_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1126: RuntimeWarning: invalid value encountered in d
    new_unnormalized_variance -= correction**2 / new_sample_count
Final feature matrix shape: (1373, 8444)
```

```
import numpy as np
```

```
print("X shape:", X.shape)
print("Any NaN in X? ", np.isnan(X).any())
print("Any non-finite X? ", ~np.isfinite(X).all())
```

```
# Which rows/cols are bad (optional):
bad_rows = ~np.isfinite(X).all(axis=1)
bad_cols = ~np.isfinite(X).all(axis=0)
print("Bad rows:", bad_rows.sum(), "Bad cols:", bad_cols.sum())
```

```
X shape: (1373, 8444)
Any NaN in X?      True
Any non-finite X?  True
Bad rows: 1373 Bad cols: 1
```

```
#4b) Rebuild preprocessing robustly (impute → drop zero-variance → scale → one-hot)

#Rebuild preprocessing robustly (impute → drop zero-variance → scale → one-hot)

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import VarianceThreshold
import numpy as np
import pandas as pd

# Start from the original df, and num_cols / cat_cols you already defined earlier
work = df.copy()

# Ensure numeric dtype
for c in num_cols:
    work[c] = pd.to_numeric(work[c], errors="coerce")

# Pipelines
num_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("var", VarianceThreshold(threshold=0.0)), # drop zero-variance numerics (prevents scaler NaNs)
    ("scaler", StandardScaler())
])

cat_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")), # fill missing categories
    ("ohe", OneHotEncoder(handle_unknown="ignore", drop="first", sparse_output=False))
])

pre = ColumnTransformer(
    transformers=[
        ("num", num_pipe, num_cols),
        ("cat", cat_pipe, cat_cols)
    ],
    remainder="drop"
)

X = pre.fit_transform(work)

# ----- Recover feature names (after VT + OHE) -----
# Numeric names after variance threshold
num_mask = pre.named_transformers_["num"].named_steps["var"].get_support()
num_names_kept = [col for col, keep in zip(num_cols, num_mask) if keep]

# Categorical names from OHE
cat_names = []
if len(cat_cols):
    cat_names = list(pre.named_transformers_["cat"].named_steps["ohe"].get_feature_names_out(cat_cols))

feature_names = num_names_kept + cat_names

# Final safety: coerce any stray non-finites (should be none after VT)
if not np.isfinite(X).all():
    print("Warning: non-finite values found post-transform. Coercing with nan_to_num.")
    X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)

print("Final feature matrix shape:", X.shape)
print("All finite?", np.isfinite(X).all())
print("Numeric features kept:", len(num_names_kept), " | Categorical features:", len(cat_names))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any obser
warnings.warn(
```

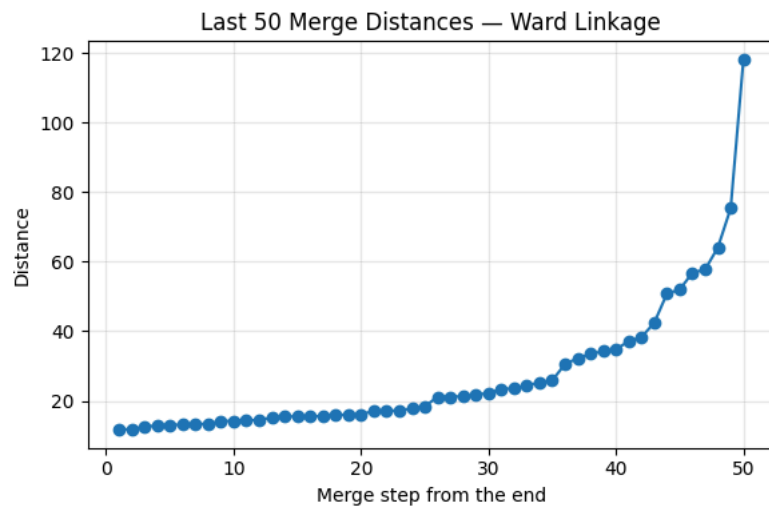
```
Final feature matrix shape: (1373, 8434)
All finite? True
Numeric features kept: 22 | Categorical features: 8412
```

#5) Linkage matrix (Z) and “last merges” plot

```
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
from pathlib import Path

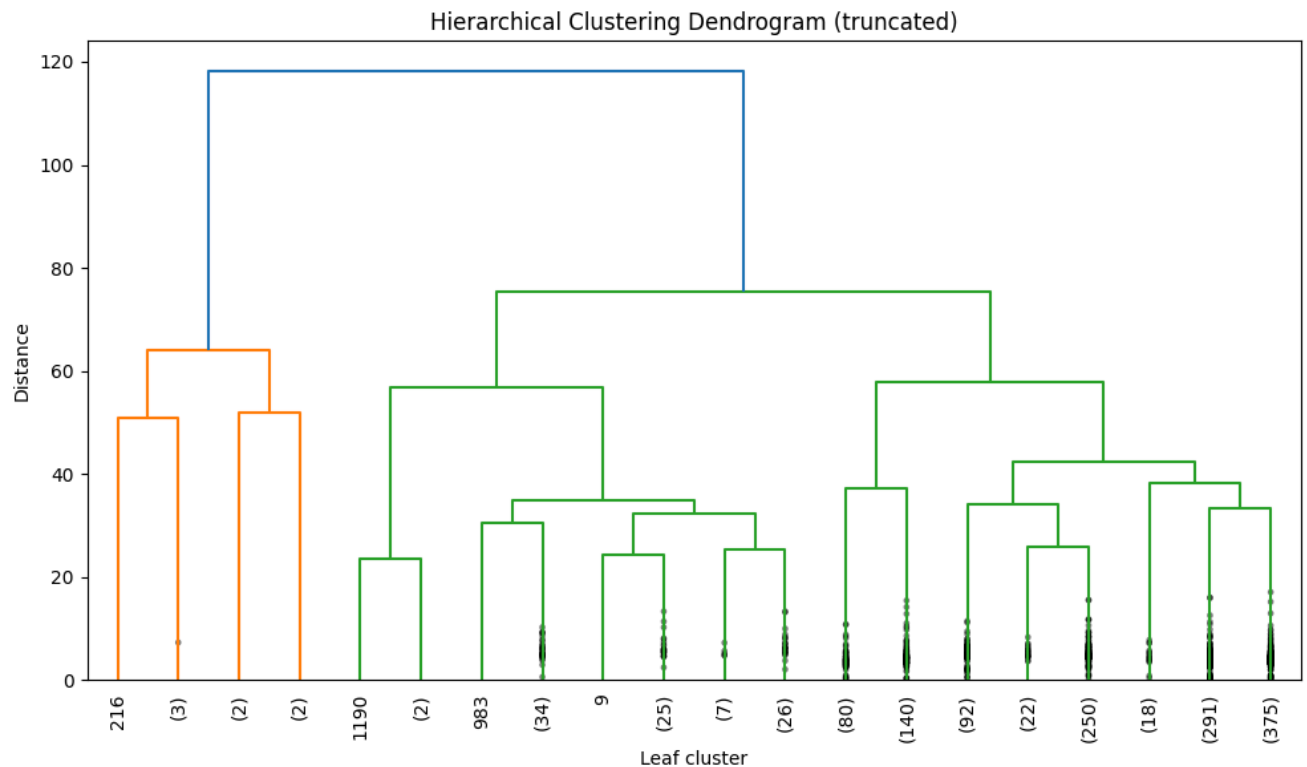
# Ward linkage on dense, finite matrix X
Z = linkage(X, method="ward")

# Inspect last 50 merge distances
plt.figure(figsize=(6,4))
last = Z[-50:, 2]
plt.plot(range(1, 51), last, marker="o")
plt.title("Last 50 Merge Distances – Ward Linkage")
plt.xlabel("Merge step from the end")
plt.ylabel("Distance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/last_50_merge_distances_ward_linkage.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



#6) Truncated dendrogram (readable)

```
plt.figure(figsize=(10,6))
dendrogram(
    Z,
    truncate_mode="lastp",
    p=20,
    leaf_rotation=90.,
    leaf_font_size=10.,
    show_contracted=True
)
plt.title("Hierarchical Clustering Dendrogram (truncated)")
plt.xlabel("Leaf cluster")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/hierarchical_clustering_dendrogram.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



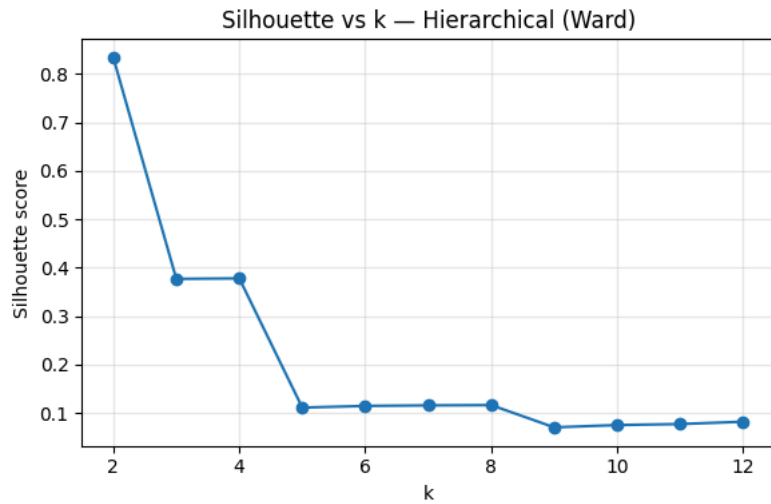
#7) Silhouette vs. k (2–12) with Agglomerative (Ward)

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

```
ks = range(2, 13)
sil_scores = []
for k in ks:
    labels_k = AgglomerativeClustering(n_clusters=k, linkage="ward").fit_predict(X)
    sil_scores.append(silhouette_score(X, labels_k))
```

```
plt.figure(figsize=(6,4))
plt.plot(list(ks), sil_scores, marker="o")
plt.title("Silhouette vs k – Hierarchical (Ward)")
plt.xlabel("k")
plt.ylabel("Silhouette score")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouetter_vs_k_hierarchical.png",
            dpi=300, bbox_inches="tight")
plt.show()
```

```
for k, s in zip(ks, sil_scores):
    print(f"k={k:2d} silhouette={s:.3f}")
```



```

k= 2  silhouette=0.834
k= 3  silhouette=0.377
k= 4  silhouette=0.378
k= 5  silhouette=0.111
k= 6  silhouette=0.115
k= 7  silhouette=0.116
k= 8  silhouette=0.117
k= 9  silhouette=0.071
k=10  silhouette=0.075
k=11  silhouette=0.077
k=12  silhouette=0.082

```

```

#Pick k_best, fit final model, sizes, PCA 2D

from sklearn.decomposition import PCA

k_best = 4 # <-- adjust based on plots above

agg_final = AgglomerativeClustering(n_clusters=k_best, linkage="ward")
labels = agg_final.fit_predict(X)

work["cluster"] = labels
print("Cluster sizes:\n", work["cluster"].value_counts().sort_index())

# PCA projection
pca2 = PCA(n_components=2, random_state=42)
proj2 = pca2.fit_transform(X)

plt.figure(figsize=(7,6))
for c in range(k_best):
    idx = labels == c
    plt.scatter(proj2[idx,0], proj2[idx,1], s=25, alpha=0.85, label=f"Cluster {c}")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(f"Hierarchical (Ward) - PCA 2D Projection (k={k_best})")
plt.legend()
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/hierarchical_pca_2d_projection.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Explained variance ratio (PC1, PC2):", pca2.explained_variance_ratio_)

```

Cluster sizes:

cluster

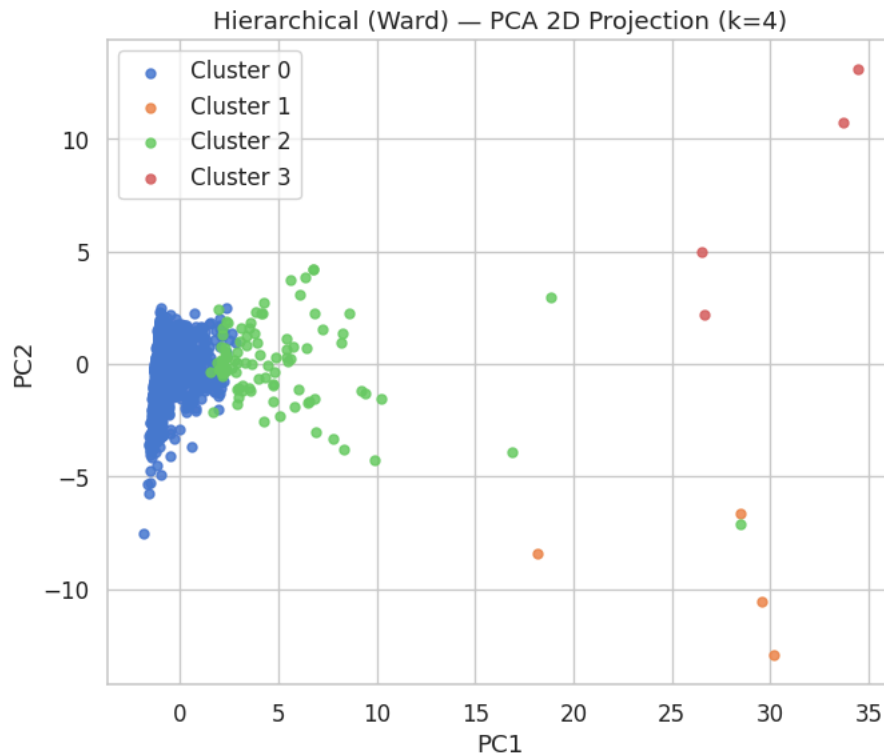
0 1268

1 4

2 97

3 4

Name: count, dtype: int64

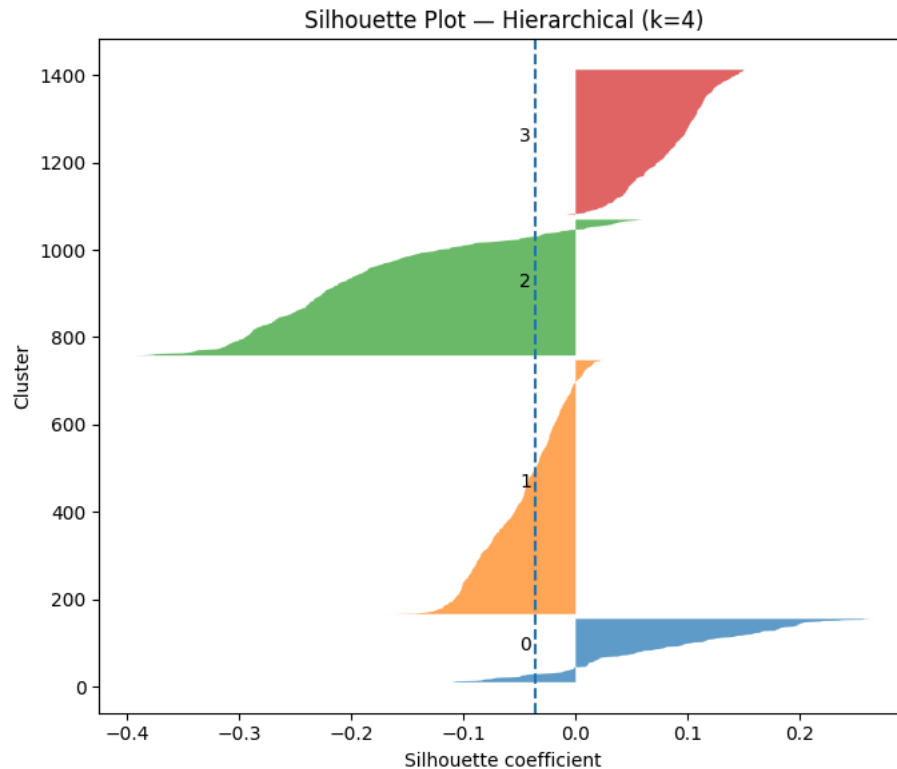


#9) Silhouette bar plot per cluster

```
from sklearn.metrics import silhouette_samples
import numpy as np

sample_sil = silhouette_samples(X, labels)
y_lower = 10
plt.figure(figsize=(7,6))
for i in range(k_best):
    ith = sample_sil[labels == i]
    ith.sort()
    size_i = ith.shape[0]
    y_upper = y_lower + size_i
    plt.fill_betweenx(np.arange(y_lower, y_upper), 0, ith, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5*size_i, str(i))
    y_lower = y_upper + 10

plt.axvline(np.mean(sample_sil), linestyle="--")
plt.xlabel("Silhouette coefficient")
plt.ylabel("Cluster")
plt.title(f"Silhouette Plot – Hierarchical (k={k_best})")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouetter_plot_hierarchical.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



```
# Numeric summaries
profile_count = work["cluster"].value_counts().sort_index().rename("count")
profile_mean = work.groupby("cluster")[num_names_kept].mean(numeric_only=True).add_suffix("_mean")
profile_med = work.groupby("cluster")[num_names_kept].median(numeric_only=True).add_suffix("_median")
profile = pd.concat([profile_count, profile_mean, profile_med], axis=1)

# Optional: top 3 categories per cluster
cat_top = {}
for c in cat_cols:
    cat_top[c] = work.groupby("cluster")[c].agg(lambda s: s.value_counts().head(3))

# Save
profile.to_csv(out_dir/"06_cluster_profile_hierarchical.csv")
labeled = df.copy()
labeled["cluster"] = labels
labeled.to_csv(out_dir/"products_with_clusters_hierarchical.csv", index=False)

print("Saved:")
print("-", out_dir/"01_last_merges.png")
print("-", out_dir/"02_dendrogram_truncated.png")
print("-", out_dir/"03_silhouette_vs_k_hierarchical.png")
print("-", out_dir/"04_pca2_hierarchical.png")
print("-", out_dir/"05_silhouette_bars_hierarchical.png")
print("-", out_dir/"06_cluster_profile_hierarchical.csv")
print("-", out_dir/"products_with_clusters_hierarchical.csv")
```

```

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'cluster'

```

The above exception was the direct cause of the following exception:

```

-----
KeyError                                Traceback (most recent call last)
_____ 2 frames _____
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
    3813         except TypeError:
    3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'cluster'

```

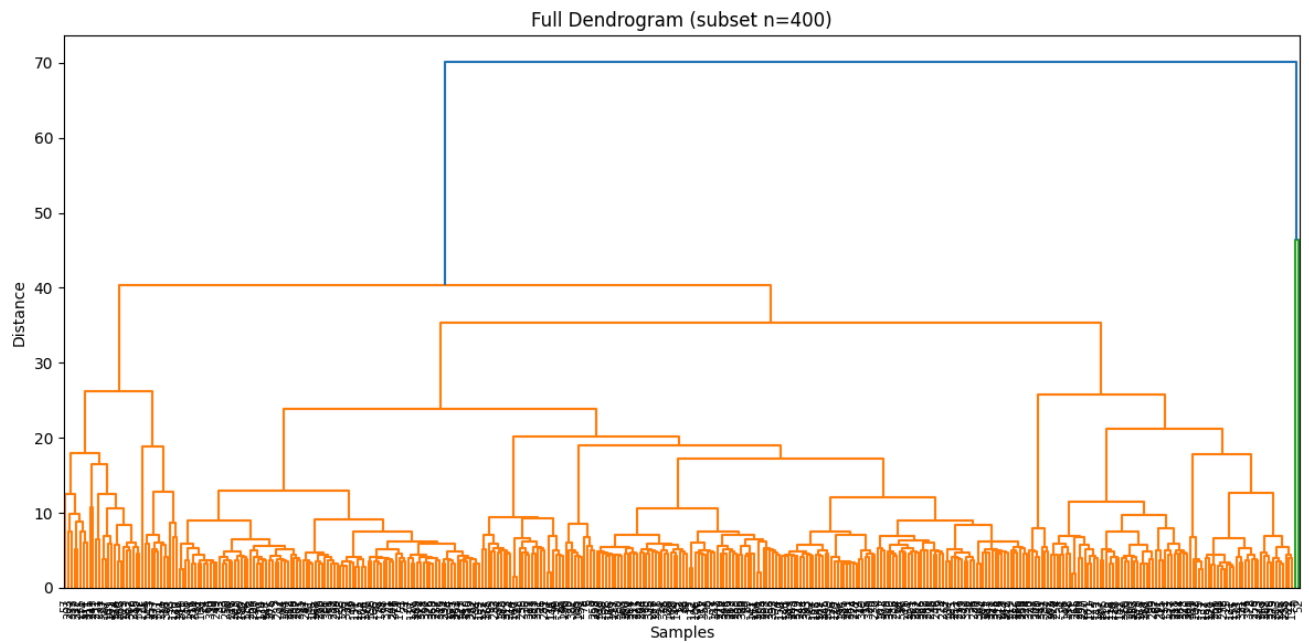
```

from scipy.cluster.hierarchy import linkage, dendrogram

subset_n = min(400, X.shape[0])
idx = np.random.choice(X.shape[0], subset_n, replace=False)
Z_sub = linkage(X[idx], method="ward")

plt.figure(figsize=(12,6))
dendrogram(Z_sub, leaf_rotation=90., leaf_font_size=7.)
plt.title(f"Full Dendrogram (subset n={subset_n})")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/full_dendrogram.png",
            dpi=300, bbox_inches="tight")
plt.show()

```



```
!pip -q install pylustering
```

Preparing metadata (setup.py) ... done 2.6/2.6 MB 37.8 MB/s eta 0:00:00
Building wheel for pylustering (setup.py) ... done

```
import pandas as pd
base = "/content/drive/MyDrive/Dat Capstone" # same path you used earlier
csv_path = f"{base}/cleaned_makeup_products.csv"

# Load your products dataset
df = pd.read_csv(csv_path, low_memory=False)

# Drop text / ID / non-numeric columns
drop_cols = [
    "product_link_id", "product_link", "item_id", "product_name",
    "description", "pros", "cons", "best_uses", "describe_yourself", "brand", "category"
]
drop_cols = [c for c in drop_cols if c in df.columns]

# Keep only numeric features
num_cols = [c for c in df.columns if c not in drop_cols]
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

numeric_df = df.loc[:, num_cols].dropna(axis=1, how="all")
numeric_df.head()
```

	price	num_shades	rating	num_reviews	review_star_1	review_star_2	review_star_3	review_star_4	review_star_5	rating
0	55.0	NaN	4.4	951.0	286	86	533	17	29	4.4
1	57.0	42.0	4.5	2234.0	35	56	123	507	1513	4.5
2	44.0	NaN	4.1	3359.0	2033	348	229	240	510	4.1
3	55.0	NaN	4.4	5071.0	226	3274	982	403	186	4.4
4	22.0	NaN	4.3	28718.0	1726	1462	2068	4730	18736	4.3

5 rows × 21 columns

```
X = numeric_df.fillna(numeric_df.median(numeric_only=True)).copy()
...
```

```
import numpy as np
from sklearn.preprocessing import StandardScaler

# Fill missing values with medians
X = numeric_df.fillna(numeric_df.median(numeric_only=True)).copy()

# Log-transform long-tailed columns (the large-count ones)
skew_cols = [c for c in X.columns if (X[c].max() > 100) and (X[c].min() >= 0)]
for c in skew_cols:
    X[c] = np.log1p(X[c])

# Standardize all numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("X_scaled shape:", X_scaled.shape)
```

X_scaled shape: (1373, 21)

```
from pyclustering.cluster.kmedoids import kmedoids
from pyclustering.utils.metric import distance_metric, type_metric
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
metric = distance_metric(type_metric.EUCLIDEAN)

def run_kmedoids_for_k(X_scaled, k, n_init=10):
    best = {"sil": -1, "labels": None, "medoids": None}
    n = X_scaled.shape[0]
    for _ in range(n_init):
        init = rng.choice(n, size=k, replace=False).tolist()
        algo = kmedoids(X_scaled.tolist(), init, metric=metric)
        algo.process()
        clusters = algo.get_clusters()
        labels = np.full(n, -1, dtype=int)
        for cid, idxs in enumerate(clusters):
            labels[idxs] = cid
        if (labels >= 0).all():
            sil = silhouette_score(X_scaled, labels)
            if sil > best["sil"]:
                best = {"sil": sil, "labels": labels, "medoids": algo.get_medoids()}
    return best

def total_cost(X_scaled, labels, medoids):
    M = X_scaled[medoids]
    d = np.sqrt(((X_scaled[:,None,:] - M[None,:,:])**2).sum(axis=2)).min(axis=1)
    return d.sum()

ks = range(2, 13)
costs, sils, best_runs = [], [], []
for k in ks:
    res = run_kmedoids_for_k(X_scaled, k, n_init=10)
    best_runs.append(res)
    costs.append(total_cost(X_scaled, res["labels"], res["medoids"]))
    sils.append(res["sil"])
```

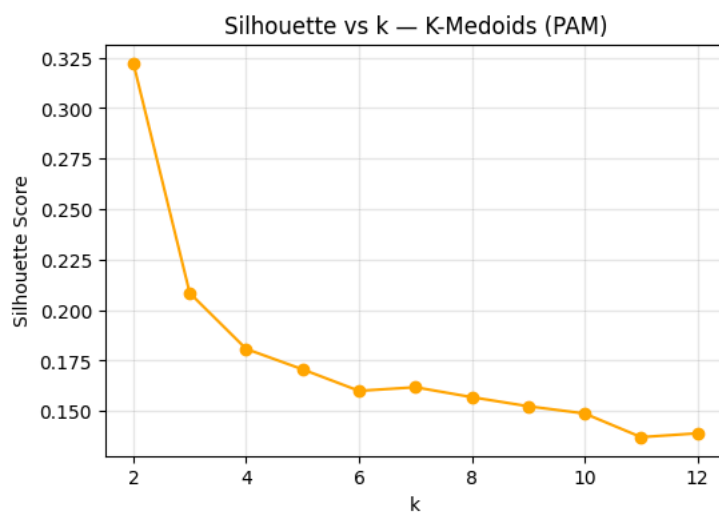
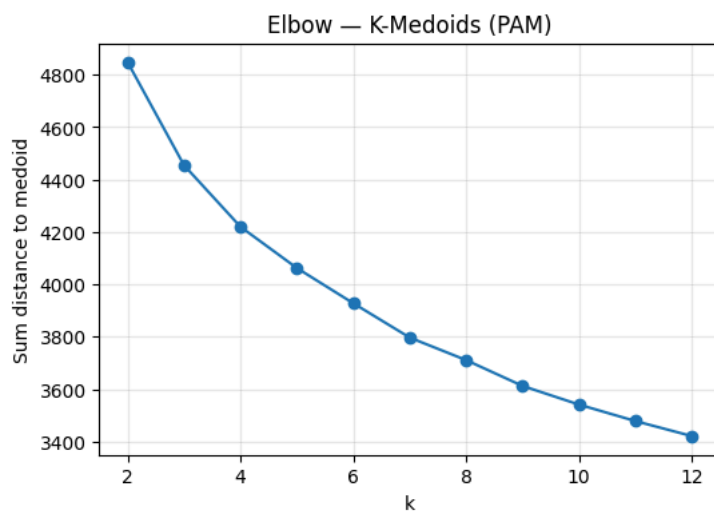
```

plt.figure(figsize=(6,4))
plt.plot(list(ks), costs, marker="o")
plt.xlabel("k"); plt.ylabel("Sum distance to medoid")
plt.title("Elbow - K-Medoids (PAM)")
plt.grid(alpha=.3);
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_elbow.png",
            dpi=300, bbox_inches="tight")
plt.show()

plt.figure(figsize=(6,4))
plt.plot(list(ks), sils, marker="o", color="orange")
plt.xlabel("k"); plt.ylabel("Silhouette Score")
plt.title("Silhouette vs k - K-Medoids (PAM)")
plt.grid(alpha=.3);
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_silhouette_vs_k.png",
            dpi=300, bbox_inches="tight")
plt.show()

for k, s in zip(ks, sils):
    print(f"k={k}, silhouette={s:.3f}")

```



```

k=2, silhouette=0.322
k=3, silhouette=0.208
k=4, silhouette=0.181
k=5, silhouette=0.171
k=6, silhouette=0.160
k=7, silhouette=0.162
k=8, silhouette=0.157
k=9, silhouette=0.152
k=10, silhouette=0.149
k=11, silhouette=0.137
k=12, silhouette=0.139

```

```
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples
import numpy as np

k_best = 4 # change this if a different k gives best silhouette
best = best_runs[k_best - ks.start]
labels, medoids = best["labels"], best["medoids"]

df_kmed = df.copy()
df_kmed["cluster"] = labels

print("Cluster sizes:\n", df_kmed["cluster"].value_counts().sort_index())

# Cluster summary
summary = df_kmed.groupby("cluster")[X.columns].mean().round(2)
display(summary)

# PCA plot
pca = PCA(n_components=2, random_state=42)
proj2 = pca.fit_transform(X_scaled)
plt.figure(figsize=(7,6))
for c in range(k_best):
    plt.scatter(proj2[labels==c,0], proj2[labels==c,1], s=30, alpha=.85, label=f"Cluster {c}")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.title(f"K-Medoids (k={k_best}) - PCA 2D Projection")
plt.legend();
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_pca2.png",
            dpi=300, bbox_inches="tight")
plt.show()
```

Cluster sizes:

cluster

0 275

1 384

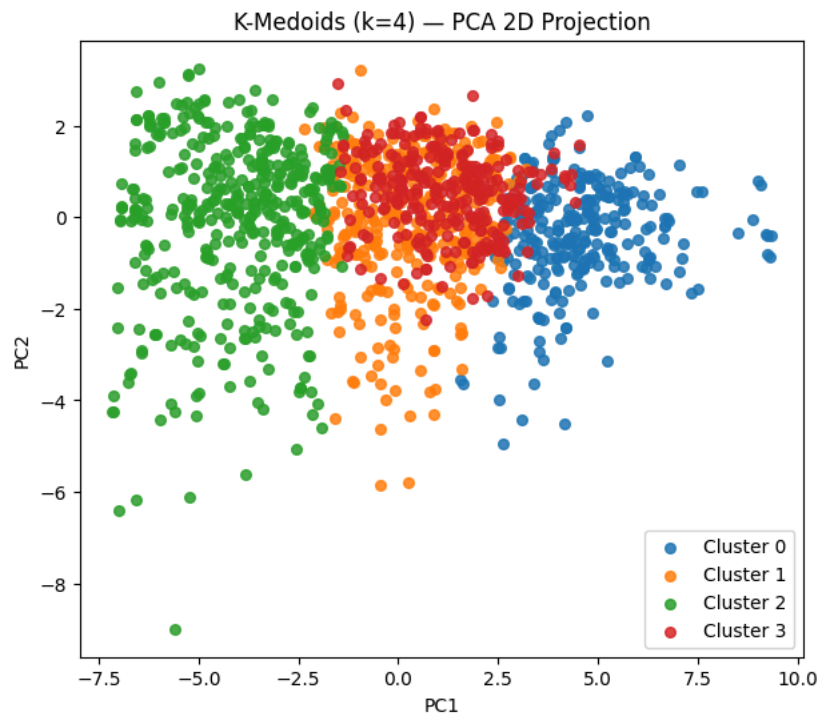
2 434

3 280

Name: count, dtype: int64

	price	num_shades	rating	num_reviews	review_star_1	review_star_2	review_star_3	review_star_4	review_star_5
cluster									
0	26.56	22.90	4.29	4024.97	913.56	789.35	605.46	662.60	1054.10
1	26.95	22.22	4.29	396.57	99.95	24.22	46.51	60.71	160.71
2	26.42	15.40	4.22	32.40	8.25	9.34	5.33	3.69	1.69
3	27.15	22.81	4.46	650.04	106.99	339.63	119.79	55.29	219.79

4 rows x 21 columns



#Show the actual medoid products (representatives).

```
df.iloc[medoids][["product_name","brand","price","average_rating"]]
```

	product_name	brand	price	average_rating
556	Liquid Lip Blush & Cheek Tint	Benefit Cosmetics	26.0	4.11
236	Shape Tape Radiant Medium Coverage Concealer	Tarte	32.0	4.28
1078	Mini Translucent Loose Setting Powder Ultra-Blur	Ask A Question	28.0	4.33
1259	Rinse-Off Eye Makeup Remover Solvent	NaN	26.0	4.49

#Profile clusters (which features separate them most).

Quick stats + effect sizes:

```
import pandas as pd, numpy as np
```

```
prof_mean = df_kmed.groupby("cluster")[X.columns].mean().round(2)
```

```
prof_med = df_kmed.groupby("cluster")[X.columns].median().round(2)
```

```
prof_iqr = df_kmed.groupby("cluster")[X.columns].quantile([.25,.75]).unstack(0)
```

```
display(prof_mean); display(prof_med); display(prof_iqr)
```

```
File "/tmp/ipython-input-516163561.py", line 2
    Quick stats + effect sizes:
    ^
SyntaxError: invalid syntax
```

--- Extended EDA ---

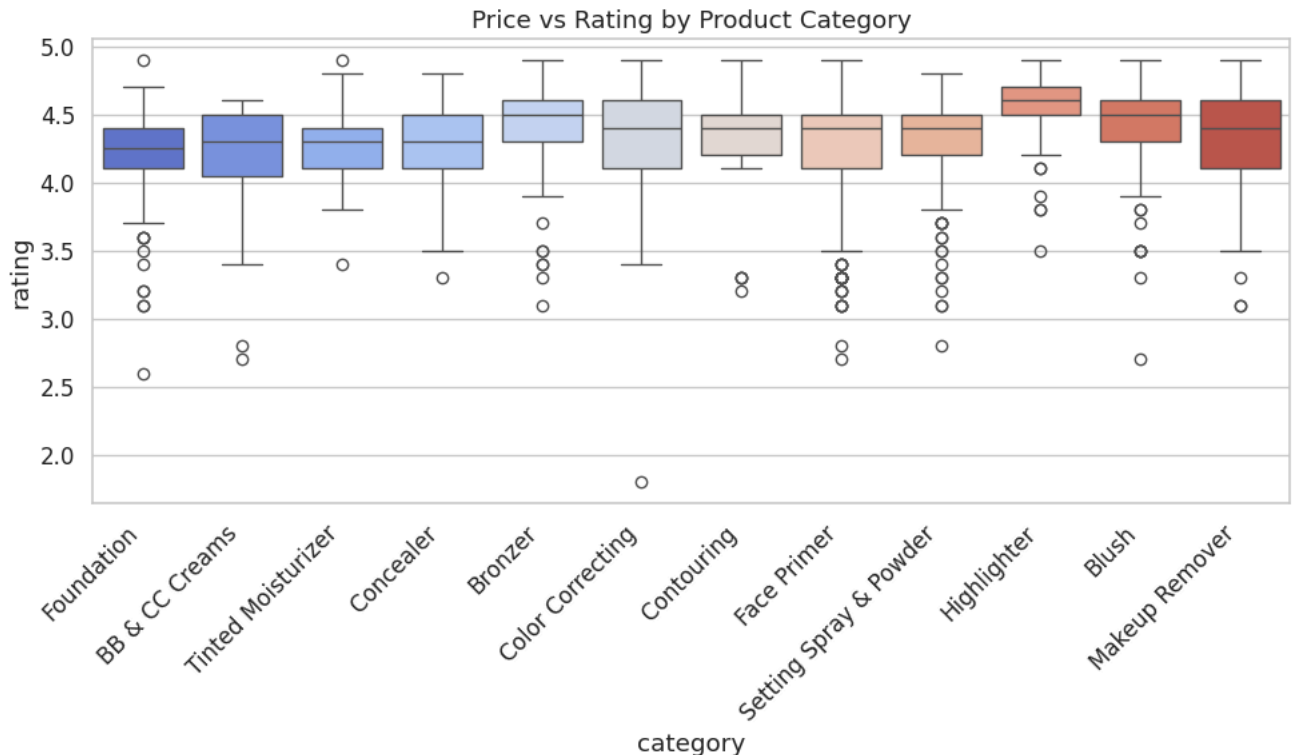
```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="whitegrid", palette="muted", font_scale=1.1)
plt.rcParams['figure.figsize'] = (8, 6)

plt.figure(figsize=(10,6))
sns.boxplot(data=products, x="category", y="rating", palette="coolwarm")
plt.xticks(rotation=45, ha='right')
plt.title("Price vs Rating by Product Category")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/price_vs_rating_by_category.png",
            dpi=300, bbox_inches="tight")
plt.show()
```

/tmp/ipython-input-440378652.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue`

```
sns.boxplot(data=products, x="category", y="rating", palette="coolwarm")
```



```
# ✅ Clean + prepare
reviews['created_date'] = pd.to_datetime(reviews['created_date'], errors='coerce')
reviews['updated_date'] = pd.to_datetime(reviews['updated_date'], errors='coerce')
reviews['year'] = reviews['created_date'].dt.year
reviews['review_length'] = reviews['comments'].astype(str).str.len()

# --- FIX DATETIME ---
for c in ['created_date', 'updated_date']:
    reviews[c] = pd.to_datetime(reviews[c], errors='coerce', infer_datetime_format=True)

# --- CREATE YEAR COLUMN ---
reviews['year'] = reviews['created_date'].dt.year
```

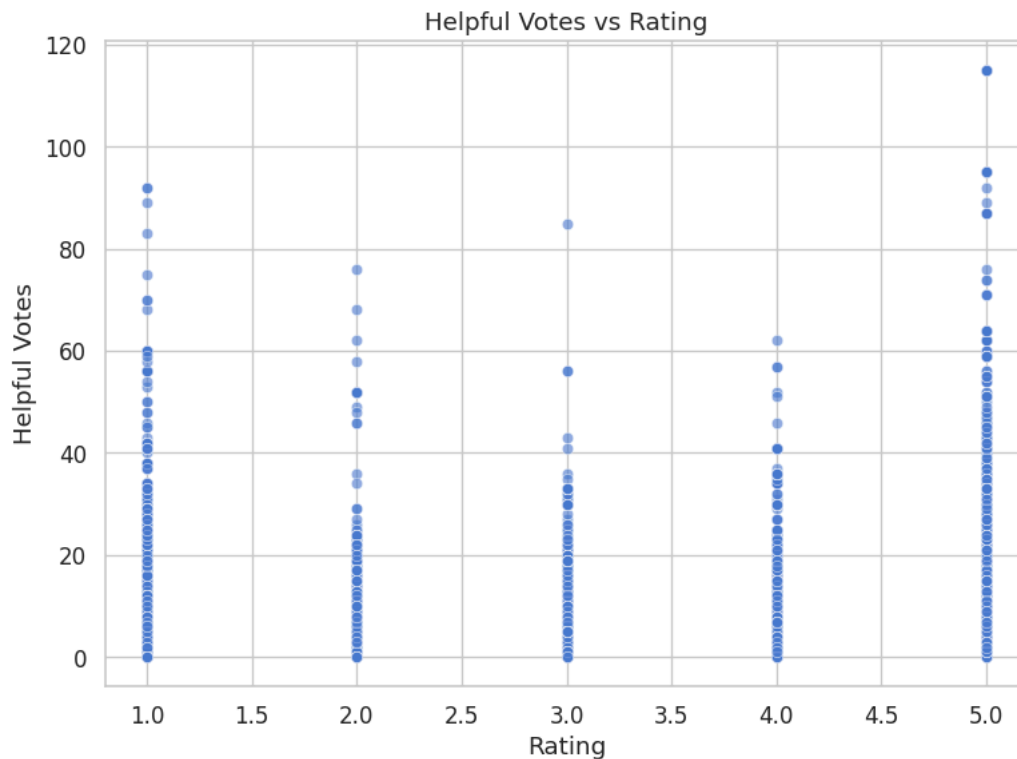
/tmp/ipython-input-682157966.py:9: UserWarning: The argument 'infer_datetime_format' is deprecated and will be removed in a future version. Please use 'format=' instead.
 reviews[c] = pd.to_datetime(reviews[c], errors='coerce', infer_datetime_format=True)
 /tmp/ipython-input-682157966.py:9: UserWarning: The argument 'infer_datetime_format' is deprecated and will be removed in a future version. Please use 'format=' instead.

```
reviews[c] = pd.to_datetime(reviews[c], errors='coerce', infer_datetime_format=True)
```

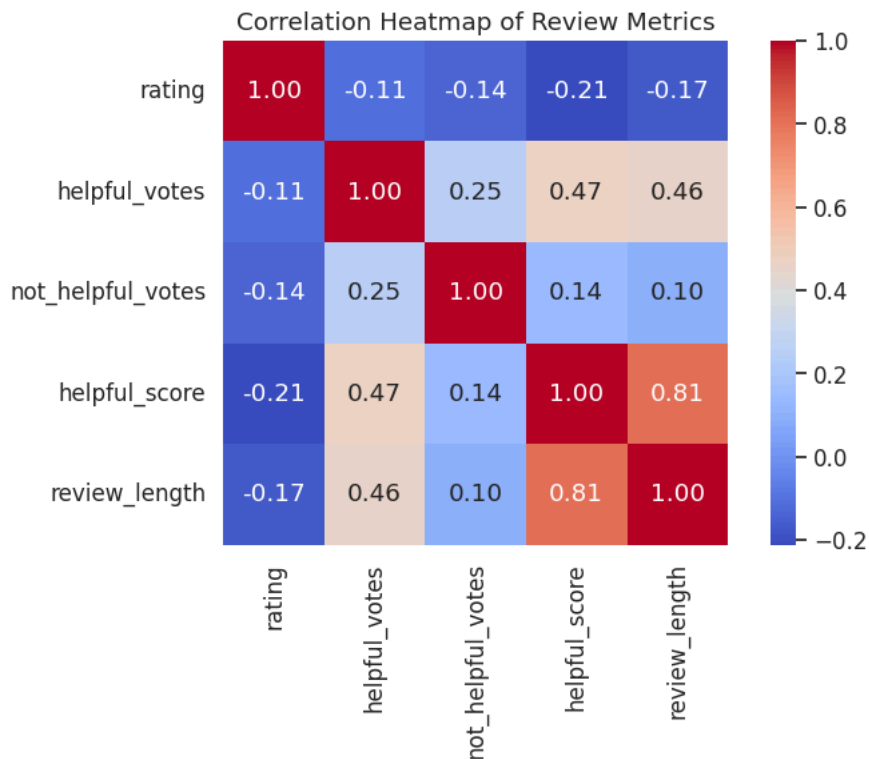
```
trend = reviews.groupby('year').size().reset_index(name='review_count')
sns.lineplot(data=trend, x='year', y='review_count', marker='o')
plt.title('Review Volume Over Time')
plt.xlabel('Year'); plt.ylabel('Number of Reviews')
plt.tight_layout();
plt.savefig("/content/drive/MyDrive/Dat Capstone/review_volume_over_time.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



```
sns.scatterplot(data=reviews, x='rating', y='helpful_votes', alpha=0.6)
plt.title('Helpful Votes vs Rating')
plt.xlabel('Rating'); plt.ylabel('Helpful Votes')
plt.tight_layout();
plt.savefig("/content/drive/MyDrive/Dat Capstone/helpful_votes_vs_rating.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



```
num_cols = ['rating', 'helpful_votes', 'not_helpful_votes', 'helpful_score', 'review_length']
corr = reviews[num_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Heatmap of Review Metrics')
plt.tight_layout();
plt.savefig("/content/drive/MyDrive/Dat Capstone/correlation_heatmap_of_review_metrics.png",
            dpi=300, bbox_inches="tight")
plt.show()
```



```
positive_text = " ".join(reviews.loc[reviews['rating']>=4, 'comments'].dropna().astype(str))
negative_text = " ".join(reviews.loc[reviews['rating']<=2, 'comments'].dropna().astype(str))
```